Behavior-Aware Crowd Simulation and Management Plugin for Unity

Project Idea

The idea of this project is to develop a reusable crowd simulation and management package for the Unity game engine. In a typical game, non-player characters (NPCs) navigate toward assigned destinations, but their movement and decisions often do not account for the wider crowd, shared resources, nearby events, or relationships between characters. Our system will extend basic navigation with configurable behavior and coordination so that agents can respond to the situation around them instead of following fixed routes and actions.

The package will use Unity AI Navigation and NavMesh as its initial movement system. Additional layers will influence destination selection, path preferences, interactions, and group decisions without requiring a complete replacement of the engine's navigation system. The goal is a general-purpose plugin that can be used in different games and simulations rather than a solution limited to one scenario.

Aim

The main aim is to make large groups of NPCs more believable and manageable by combining crowd-aware navigation with character behavior, perception, emotional state, and social interactions. We also want to investigate how these systems affect one another, such as when a stressful event changes agent decisions, causes a queue to grow, and requires the crowd manager to redistribute traffic. The result should remain configurable, explainable, and efficient enough for real-time simulation.

Project Scope

The core project will be a modular Unity C# package containing agent components, resource-point management, navigation and congestion control, state-based behavior, perception, emotional influence, group coordination, and debugging tools. The navigation work will focus on a crowd-aware routing layer over standard NavMesh paths, potentially using regions or an auxiliary graph with congestion-dependent costs. Creating an entirely new NavMesh generation or physics engine is outside the scope of the project.

We will also build a shared 3D simulation environment with reusable spawn zones, goal zones, obstacles, and controlled test scenarios. This environment will provide a consistent place for the team to develop and compare features; it is a testing ground underneath the plugin, not the plugin itself. LLM-based state transitions and high-level congestion decisions are considered optional extensions after the core systems are functional, rather than dependencies of normal agent behavior.

Currently Used Technologies

Unity AI Navigation already supports NavMesh-based pathfinding, dynamic obstacles, and agent movement [1]. Established crowd-navigation methods include local collision avoidance and cooperative pathfinding, which address movement conflicts between agents [2][3]. These methods provide useful foundations, but game-specific behavior such as resource usage, emotional responses, group relationships, and congestion management is generally implemented as additional logic. Our project will use existing navigation technology and concentrate on integrating these higher-level concerns into one reusable package.

Proposed Functionalities

At the navigation level, agents will consider not only the distance to a destination but also crowd density, bottlenecks, and the availability of alternatives. Developers will be able to define resource points such as workstations, shops, exits, or rest areas with limited capacity and interaction times. Agents will select resources, wait or queue when necessary, and reconsider their choice when travel or waiting costs become too high. A crowd manager will monitor traffic and resource usage and adjust route preferences or redistribute demand when congestion develops.

At the individual level, characters will have configurable traits, goals, and behavioral states. Events such as an alarm, a blocked passage, or an overcrowded area can trigger state changes that affect speed, destination choice, patience, or willingness to follow others. Agents will perceive nearby objects, events, and characters through a configurable field of view, range, and optional line-of-sight checks. Emotional influence can propagate between nearby agents under controlled rules, allowing local reactions to affect the broader crowd without becoming unpredictable.

At the social level, agents will be able to form groups with shared destinations, leaders, and cohesion rules. Groups may travel together, separate around obstacles, and regroup afterward. Unity Editor settings and runtime debug views will expose paths, perception ranges, agent states, resource occupancy, congestion, and the reasons behind selected decisions. These features will make it possible to test individual modules separately and understand how they behave when combined.

What We Will Do Differently (Innovation)

The main contribution will be the interaction between systems that are commonly developed as separate game-specific features. Navigation, resource points, perception, emotional reactions, group behavior, and crowd management will share a consistent representation of agent state and world events. For example, an agent may perceive a threat, change its behavior, influence nearby characters, choose a different exit, and contribute to congestion that the crowd manager then detects and responds to. We will investigate whether this combined approach produces more coherent crowd behavior than independent NavMesh agents and isolated scripted rules.

As an experimental extension, an LLM may recommend transitions between predefined behavior states or propose high-level congestion-management actions. These outputs would be restricted to valid actions and compared against deterministic rules. Per-frame LLM control of each agent is not part of the planned core implementation.

Evaluation and Deliverables

We will compare a standard NavMesh baseline with progressively enabled plugin modules in repeatable scenarios involving opposing crowd flows, alternative corridors, competing resource points, event-triggered reactions, and group movement. Measurements will include travel and waiting times, destination completion, throughput, congestion duration, and frame time. The expected deliverables are a documented Unity package, reusable agent and resource prefabs, a shared 3D testing environment, debugging utilities, and a demonstration with comparative results.

References

[1] Unity Technologies, "AI Navigation," Unity Manual.

[2] J. van den Berg et al., "Optimal Reciprocal Collision Avoidance," 2010.

[3] D. Silver, "Cooperative Pathfinding," AIIDE, 2005.